

Constraint-Based Problem Solving

Intelligent Exam Scheduling System

A.PEAS Description of the Exam-Scheduling Agent

PEAS stands for Performance Measure, Environment, Actuators, and Sensors.

Component	Description
Performance Measure	Generate a valid timetable, satisfy all course-conflict constraints, and schedule all five examinations within three slots.
Environment	University examination system containing five courses, three examination slots, and conflicting student enrollments.
Actuators	Assign or change an examination slot for a course and generate the final timetable.
Sensors	Identify courses, available slots, existing assignments, and conflicting course pairs.

Performance Measure

- Schedule all five courses.
- Ensure conflicting courses receive different slots.
- Avoid timetable conflicts.
- Produce a valid timetable using the available three slots.

Environment

The environment consists of five courses and three available examination slots. Some courses share students, so they cannot have examinations at the same time.

Actuators

The scheduling agent assigns Slot1, Slot2, or Slot3 to each course. If a conflict occurs, it can change an assignment and try another slot.

Sensors

The agent checks the assigned slots, unassigned courses, and conflict relationships to determine whether an assignment is valid.

B.Goal-Based Agent

The most suitable intelligent agent for this task is a Goal-Based Agent.

The scheduling agent has a clearly defined goal: to create a valid examination timetable for all five courses using three available slots while satisfying every course-conflict constraint.

The agent must consider the courses, available slots, and conflicting pairs before making assignments. It must ensure that courses sharing students are placed in different slots.

For example, courses A and B cannot be scheduled in the same slot because students are enrolled in both courses. Similarly, courses B and C must receive different slots.

The agent can use a search strategy to explore possible slot assignments. If an assignment violates a constraint, the agent can reconsider the assignment and select another slot.

A simple reflex agent is less suitable because this problem requires considering multiple course assignments and their relationships. A model-based agent can maintain the current timetable and constraints, but the primary objective is to achieve a specified goal. A utility-based agent is not necessary because the main requirement is to find a valid timetable rather than maximize a separate utility score.

Justification

1. The agent has a clearly defined goal.
2. It must assign slots to all five courses.
3. It must satisfy all course-conflict constraints.
4. It must consider the effects of assignments on other courses.
5. It can use backtracking to correct invalid assignments.
6. It must find a valid solution within the three available slots.

C.Constraint Satisfaction Problem (CSP) Formulation

A Constraint Satisfaction Problem consists of Variables, Domains, and Constraints. In this problem, each course must be assigned an examination slot without violating the specified conflicts.

1. Variables

Each course is represented by a variable.

$$X = \{A, B, C, D, E\}$$

Where:

- A – Artificial Intelligence
- B – Database Systems
- C – Computer Networks
- D – Operating Systems
- E – Software Engineering
-

2. Domains

Each course can be assigned one of three available examination slots.

$$\begin{aligned} D(A) &= D(B) = D(C) = D(D) = D(E) \\ &= \{\text{Slot1}, \text{Slot2}, \text{Slot3}\} \end{aligned}$$

3. Constraints

Conflicting courses must be assigned different examination slots.

$$A \neq B, A \neq C, B \neq C$$

$$B \neq D, C \neq D, C \neq E, D \neq E$$

For example, if course A is assigned Slot1, course B and course C cannot be assigned Slot1.

4. Constraint Graph

The constraint graph represents courses as vertices and conflicts as edges.

Vertices:

$$V = \{A, B, C, D, E\}$$

Edges:

$$E = \{(A, B), (A, C), (B, C), (B, D), (C, D), (C, E), (D, E)\}$$

D.Plain Backtracking Search

Plain Backtracking Search assigns a slot to one course at a time. If an assignment violates a constraint, the algorithm rejects that value and tries the next available slot. If all values for a variable fail, it returns to the previous variable and changes its assignment.

The required variable order is:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$$

The domain value order is:

$$\text{Slot1} \rightarrow \text{Slot2} \rightarrow \text{Slot3}$$

Step-by-Step Assignment Table

Step	Course	Attempted Slot	Outcome	Current Assignment
1	A	Slot1	Success	A = Slot1
2	B	Slot1	Failure: conflicts with A	A = Slot1
3	B	Slot2	Success	A = Slot1, B = Slot2
4	C	Slot1	Failure: conflicts with A	A = Slot1, B = Slot2

5	C	Slot2	Failure:conflicts with B	A = Slot1, B = Slot2
6	C	Slot3	Success	A = Slot1, B = Slot2, C = Slot3
7	D	Slot1	Success	A = Slot1, B = Slot2, C = Slot3, D = Slot1
8	E	Slot1	Failure: conflicts with D	Previous assignments unchanged
9	E	Slot2	Success	All courses assigned

Explanation

1. Course A is assigned Slot1.
2. Course B cannot use Slot1 because it conflicts with A. It is assigned Slot2.
3. Course C cannot use Slot1 because it conflicts with A or Slot2 because it conflicts with B. It is assigned Slot3.
4. Course D can use Slot1 because its conflicting courses B and C use Slot2 and Slot3, respectively.
5. Course E cannot use Slot1 because it conflicts with D. It is assigned Slot2, which satisfies its constraints with C and D.

Final Valid Timetable

Course	Subject	Examination Slot
A	Artificial Intelligence	Slot1
B	Database Systems	Slot2
C	Computer Networks	Slot3
D	Operating Systems	Slot1
E	Software Engineering	Slot2

Assignment Statistics

- Total assignment attempts: 9
- Failed value attempts: 4
- Successful assignments: 5
- Variable-level backtracks: 0

In this solution, the algorithm rejects four slot values but never needs to undo an earlier successful course assignment.

E.Backtracking Search with Forward Checking

Forward Checking improves Plain Backtracking Search by updating the domains of unassigned variables after each successful assignment.

When a course is assigned a slot, that slot is removed from the domains of its unassigned conflicting courses. If any unassigned course has an empty domain, the algorithm detects failure immediately and backtracks.

The variable order remains $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$, and the domain values are considered in the order $\text{Slot1} \rightarrow \text{Slot2} \rightarrow \text{Slot3}$.

Step-by-Step Domain Pruning

Initially, all courses have the following domain:

$$\begin{aligned} D(A) &= D(B) = D(C) = D(D) = D(E) \\ &= \{S, 1, S2S3\} \end{aligned}$$

Here, S1, S2, and S3 represent Slot1, Slot2, and Slot3.

Step	Assignment	Domain Changes After Forward Checking
1	A = Slot1	Remove Slot1 from B and C. B = , C = .
2	B = Slot2	Remove Slot2 from C and D. C = , D = .
3	C = Slot3	Remove Slot3 from D and E. D = , E = .
4	D = Slot1	Remove Slot1 from E. E = .
5	E = Slot2	All constraints satisfied.

Final Valid Timetable

Course	Subject	Examination Slot
A	Artificial Intelligence	Slot1
B	Database Systems	Slot2
C	Computer Networks	Slot3
D	Operating Systems	Slot1
E	Software Engineering	Slot2

Assignment Statistics

- Successful assignment attempts: 5
- Failed assignment attempts: 0
- Variable-level backtracks: 0

Forward Checking removes inconsistent slot values before they are tried. In this particular problem, it finds a valid solution without any failed assignment attempts.

Program:

co1-At2.AI.code.py - C:/Users/SRINATH/Downloads/co1-At2.AI.code.py (3.12.10)

File Edit Format Run Options Window Help

```
courses = ["A", "B", "C", "D", "E"]
slots = ["Slot1", "Slot2", "Slot3"]

conflicts = [
    ("A", "B"), ("A", "C"), ("B", "C"),
    ("B", "D"), ("C", "D"), ("C", "E"), ("D", "E")
]

def valid(course, slot, assignment):
    for x, y in conflicts:
        if course == x and assignment.get(y) == slot:
            return False
        if course == y and assignment.get(x) == slot:
            return False
    return True

def backtracking():
    assignment = {}
    attempts = [0]
    failures = [0]

    def solve(i):
        if i == len(courses):
            return True

        course = courses[i]

        for slot in slots:
            attempts[0] += 1

            if valid(course, slot, assignment):
                assignment[course] = slot

                if solve(i + 1):
                    return True

                del assignment[course]
            else:
                failures[0] += 1

        return False

    solve(0)
    return assignment, attempts[0], failures[0]

def forward_checking():
    domains = {c: slots[:] for c in courses}
```

```
co1-At2.AI.code.py - C:/Users/SRINATH/Downloads/co1-At2.AI.code.py (3.12.10)
File Edit Format Run Options Window Help
    for slot in domains[course][:]:
        attempts[0] += 1

        if not valid(course, slot, assignment):
            failures[0] += 1
            continue

        assignment[course] = slot
        old_domains = {c: domains[c][:] for c in courses}

        for x, y in conflicts:
            if x == course and y not in assignment:
                domains[y] = [s for s in domains[y] if s != slot]
            elif y == course and x not in assignment:
                domains[x] = [s for s in domains[x] if s != slot]

        if all(domains[c] for c in courses if c not in assignment):
            if solve(i + 1):
                return True

        else:
            failures[0] += 1

        assignment.pop(course)
        for c in courses:
            domains[c] = old_domains[c]

    return False

solve(0)
return assignment, attempts[0], failures[0]

plain, p_attempts, p_failures = backtracking()
forward, f_attempts, f_failures = forward_checking()

print("Plain Backtracking Search:")
for c in courses:
    print(c, ":", plain[c])
print("Assignment Attempts:", p_attempts)
print("Failed Attempts:", p_failures)

print("\nBacktracking with Forward Checking:")
for c in courses:
    print(c, ":", forward[c])
print("Assignment Attempts:", f_attempts)
print("Failed Attempts:", f_failures)
```

Output:

```
IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/SRINATH/Downloads/co1-At2.AI.code.py =====
Plain Backtracking Search:
A : Slot1
B : Slot2
C : Slot3
D : Slot1
E : Slot2
Assignment Attempts: 9
Failed Attempts: 4

Backtracking with Forward Checking:
A : Slot1
B : Slot2
C : Slot3
D : Slot1
E : Slot2
Assignment Attempts: 5
Failed Attempts: 0
>>> |
```

F.Comparison of Plain Backtracking Search and Backtracking with Forward Checking

Criteria	Plain Backtracking	Forward Checking
Assignment Attempts	9	5
Failed Assignment Attempts	4	0
Variable-Level Backtracks	0	0
Domain Checking	Checks constraints against assigned courses	Checks constraints and prunes future domains
Search Effort	Higher	Lower
Efficiency	Less efficient for larger CSPs	More efficient when pruning eliminates many values

Note: Assignment attempts count the slot values actually tested. Variable-level backtracking means undoing a previously successful variable assignment because the remaining search cannot produce a solution. Neither method needs to undo a successful assignment in this particular run.

Forward Checking reduces search effort by removing invalid slot values from the domains of unassigned courses immediately after a successful assignment.

For example, when A = Slot1, Slot1 is removed from the domains of B and C because both conflict with A. When B = Slot2, Slot2 is removed from C and D. This helps the algorithm identify available slots without repeatedly testing values that are already known to be invalid.

The advantages of Forward Checking are:

1. It detects possible conflicts earlier.
2. It reduces the number of slot values that must be tested.
3. It prevents many unnecessary assignments.

4. It maintains the domains of unassigned courses.
5. It reduces search effort as the problem becomes larger.
6. It can detect an empty domain before proceeding further.

Conclusion of Comparison

Both algorithms produce the same valid timetable for this problem. However, Plain Backtracking makes 9 assignment attempts, whereas Forward Checking makes 5 assignment attempts. Therefore, Forward Checking is more efficient for this particular execution because it reduces the number of tested values from 9 to 5.